

> SKN20 4th Project

PaperSnack

AI 논문 트렌드 검색 챗봇

🔗 3rd → 4th 버전 고도화

🚢 Docker 기반 원클릭 배포

TEAM NAME

H **해조** (Team Haejo)

김지은

박다정

오학성

정소영

황수현

2025.01.12

AI 논문 트렌드 탐색의 새로운 기준, PaperSnack



문제 정의

PROBLEM

매주 쏟아지는 수천 편의 AI/ML 논문들 속에서,
연구자들은 최신 트렌드를 파악하기 어렵습니다.
영어 원문의 장벽과 방대한 정보량으로 인해
핵심 인사이트를 놓치기 쉽습니다.



해결 방법

SOLUTION

HuggingFace Weekly Papers 13개 주차 수집과
RAG 기반 대화형 검색을 결합했습니다. 최신 논문만을 선별하여
질문에 대한 정확한 답변과 출처를 제공합니다.

핵심 가치 (Core Values)



01

Django 웹 프레임워크 추가

Django(8000 port) 웹 애플리케이션, 사용자
인증, 채팅 히스토리 관리
FastAPI(8001 port) RAG 시스템 API 서버



02

자연어 대화 검색

"요즘 뜨는 LLM 논문 알려줘" 처럼
자연스럽게 질문하면
한글로 요약 답변 제공



03

정확도 고도화

하이브리드 검색(Vector + BM25)과 Cross-
Encoder Rerank으로 검색 품질 대폭 향상



BEST

Docker 원클릭 배포

복잡한 환경 설정 없이
docker-compose up
명령어 하나로 즉시 실행

주요 개선사항 한눈에 보기

3rd(PoC)에서 4th(Production) 버전으로의 핵심 진화 포인트

개선 항목	3RD VERSION (BEFORE)	→	4TH VERSION (AFTER)	효과 (IMPACT)
 배포 시스템	수동 설정 30분 환경 의존성 문제 발생	>	Docker 원클릭 1~3분 <code>docker-compose up</code>	 -96%
 RAG 품질	기본 Vector hybrid검색	>	Hybrid + Rerank	 RAGAS ↑
 인프라	FastAPI 단독	>	Django + Nginx + FastAPI	 실무급
 문서화	README 1개	>	상세 가이드 5개	

 RAGAS 메트릭을 통한 정량적 성능 측정 결과는 슬라이드 12에서 상세히 다룹니다.

MAJOR UPGRADE

Docker 기반 배포 자동화



개발/배포 환경의 완벽한 일치

Before (3rd)

Manual Setup

- ✖ 수동 설정 30분 소요
Python, 가상환경, 의존성 설치 반복
- ⚠ 환경 불일치 오류
"내 컴퓨터에선 되는데..." (OS 차이)



-96% 시간 단축

After (4th)

Docker Compose



배포 시간 1~3분
원클릭 자동화 & 환경 일치

deployment

```
docker compose -f compose.yaml pull  
  
docker-compose up -d --build  
  
[+] Running 3/3  
✓ Container db Started  
✓ Container backend Started  
✓ Container rag_api Started
```

Backend Service

Django + Gunicorn + Nginx

RAG AI Service

FastAPI + Uvicorn



Health Check & Auto Restart



Volume Persistence (DB/Logs)



Internal Docker Network

실무 수준의 웹 인프라 구축

★ 가장 큰 개선사항 : Nginx + Django 도입으로 안정성과 확장성 확보

BEFORE

FastAPI 단독 운영

- ✗ 단순 개발 서버 (Uvicorn) 의존
- ✗ 사용자 관리/인증 기능 부재
- ✗ 정적 파일 처리 비효율



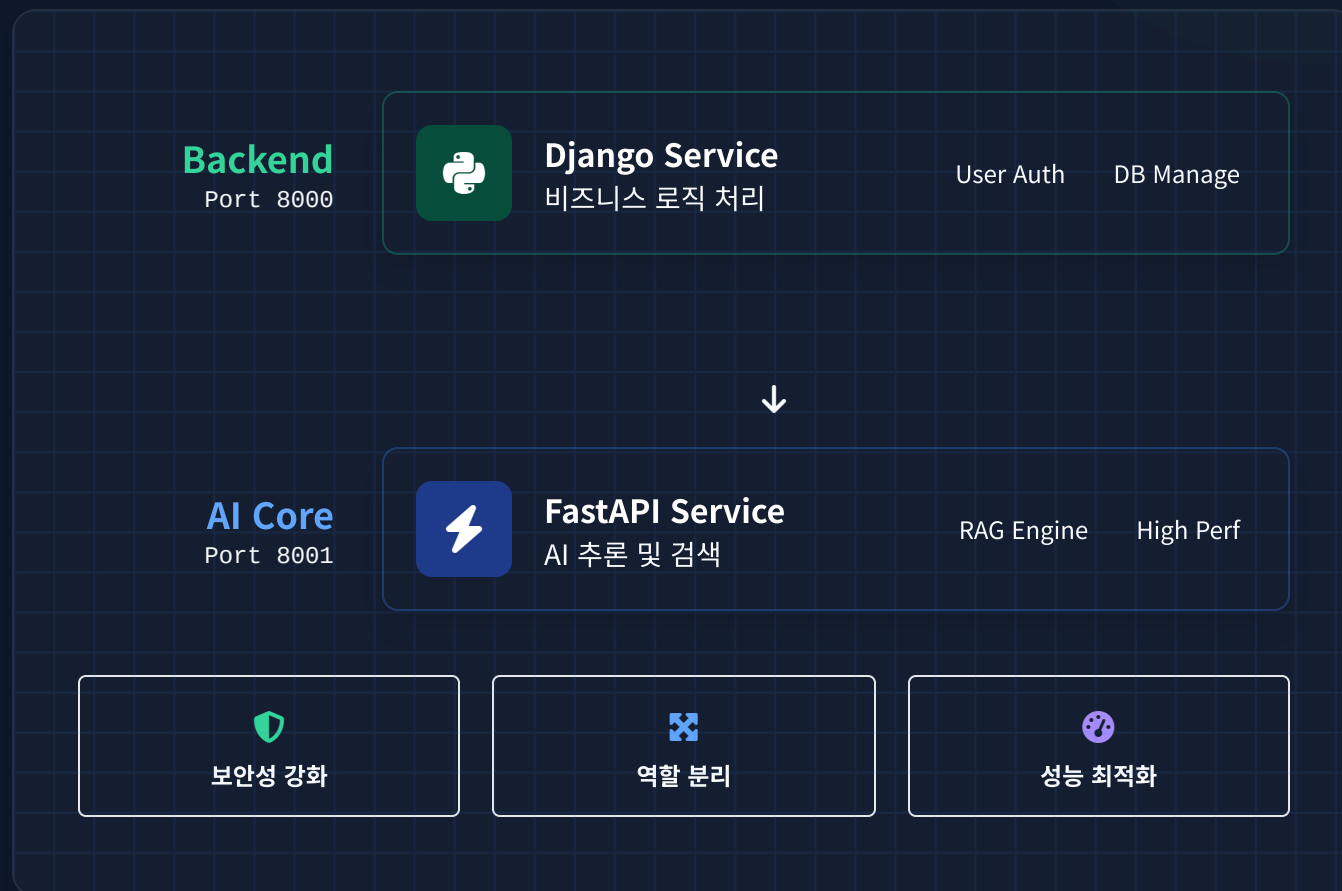
Django + Nginx 도입

Production Ready

 **Django Framework**
사용자 인증, 채팅 히스토리, ORM

 **Nginx & Gunicorn**
리버스 프록시, Gzip 압축, WSGI (Workers=3)

마이크로서비스 아키텍처 패턴



✓ RAGAS 평가로 검증된 4가지 핵심 기술 적용



01

하이브리드 검색

Vector(의미) + BM25(키워드) 결합
RRF Fusion 알고리즘으로 점수 통합 및 최적화

Effect

↑ Context Recall



02

메타데이터 부스팅

논문 제목에 키워드 포함 시 **가중치(+0.05)** 부여
단순 내용 일치보다 핵심 주제 논문 우선 노출

Effect

🎯 Relevance ↑



03

CORE TECH

Cross-Encoder Rerank

BAAI/bge-reranker-large 모델 도입
LLM 재랭킹 대비 10배 빠른 속도로 정밀도 향상

Effect

🕒 Faithfulness ↑



04

출처 최적화

컨텍스트 논문 수 **5개 → 3개**로 제한
불필요한 정보 제거로 할루시네이션 최소화

Effect

📄 Token -40%

데이터 처리 최적화

청킹(Chunking) 제거를 통한 문맥 보존 및 처리 속도 향상



Before (3rd Ver.)

FIXED CHUNKING

- ✗ 논문 초록을 **300자 단위**로 강제 분할
- ✗ 45자 오버랩 설정으로 데이터 중복 발생
- ⚠ 결과: 문장 중간이 잘려 문맥 분리 및 의미 손실



After (4th Ver.)

FULL CONTEXT

- ✓ 논문 초록 **전체**를 **하나의 문서**로 활용
- ✓ 요약문 특성상 길이가 적절하여 분할 불필요
- ★ 완결된 문맥 보존으로 검색 정확도 상승

개선 효과 (Impact)



문맥 완전성 보존

문장이 중간에 끊기지 않아
LLM이 논문의 핵심 내용을
정확하게 이해하고 요약 가능



파이프라인 단순화

복잡한 청킹/오버랩 계산 로직 제거
전처리 과정 간소화로
코드 유지보수성 향상



처리 속도 향상

불필요한 데이터 증식 방지
벡터 임베딩 연산량 감소로
전체 시스템 효율 증가



데이터 준비 시간

5-10분 → 3-5분

↓ 40% 단축

시스템 아키텍처

Docker Compose 기반의 마이크로서비스 데이터 흐름

SOURCE

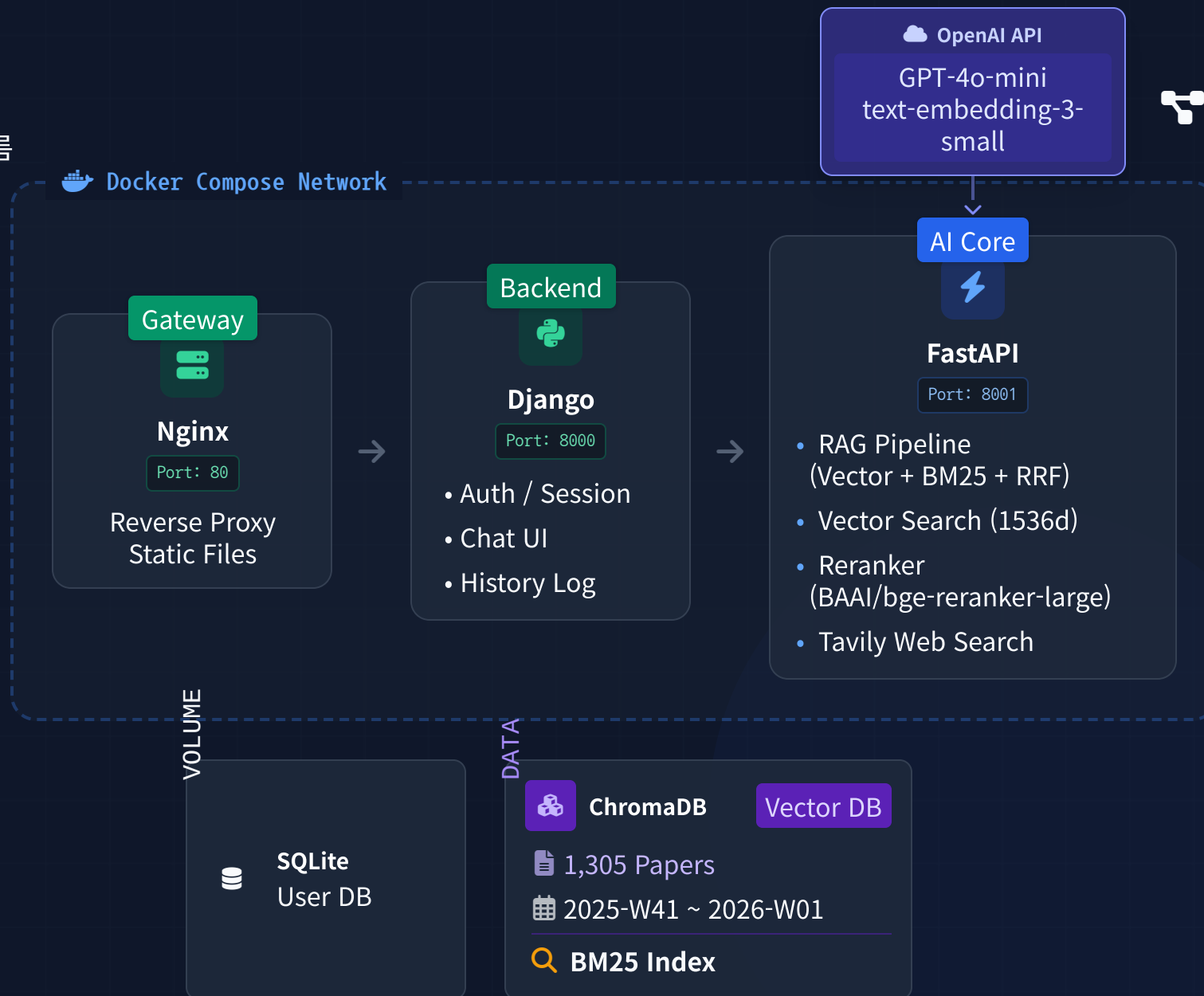


**HuggingFace
Weekly Papers**

1,305 Papers



User
Browser





6단계 노드로 구성된 정밀 검색 및 생성 프로세스

NODE 01



번역 노드

Translate

한글 질의 → 영어로 변환

Query Norm

Term Mapping

NODE 02



주제 검증

Topic Guard

AI/ML 관련성 판별 필터

✕ Web Search

✓ Pass

NODE 03



검색 노드

Hybrid Retrieve

Vector(3) + BM25(3) →

RRF Fusion(10) →

Re-ranking(3) ✓

NODE 04



평가 노드

Evaluate

문서 관련성 및 충분성 평가

Re-Retrieve?

Quality Check

NODE 05



생성 노드

Generation

GPT-4o-mini 한글 답변 생성

Prompt Eng

Formatting

NODE 06



최종 응답

Final Answer

출처 포함 답변 사용자 전달

” Sources (3) Attached



Pre-processing



Core RAG



Evaluation



Generation

프로젝트 구조 개선

가독성 향상 및 모듈화를 위한 디렉터리 구조 전면 개편

3rd Version (Before)

복잡한 번호 체계 & 단일 파일 의존

```

02_src/
├── 01_data_collection/
├── 02_utils/
├── 03_rag/
│   └── langgraph_hybrid.py (600+ lines)
├── 04_ui/
└── 05_explain/
    
```

⚠ 유지보수 어려움



4th Version (After)

모듈화 & 역할별 명확한 분리

```

src/
├── utils/
├── rag/
│   ├── prompts.py
│   ├── rag_system.py
│   └── api.py
├── backend/ (Django App)
└── docker-compose (Deployment)
    
```

KEY IMPACT



신규 개발자 온보딩 시간

폴더 구조 직관성 향상으로 적응 속도 개선

↓ 50%

사용자 친화적 웹 UI와 구조화된 답변 시스템

Web UI Features

채팅 히스토리

과거 대화 목록, 클릭 시 전체 보기

프로젝트 관리

채팅 분류/정리, 폴더별 그룹화

사용자 인증

로그인/회원가입, 프로필 관리

실시간 통계

총 논문 수

PaperSnack

HISTORY

최신 RAG(Retrie...

LLM에서 긴 문맥...

RPT와 LVLMS (l...

IRAG-4와 SoK---



하이브리드 검색

SAM 3 모델은 개념 프롬프트 기반으로 이미지와 비디오에서 객체를 감지, 분할 및 추적하는 통합 모델입니다.

주요 통찰

- SAM 3은 400만 개의 고유 개념 레이블을 포함한 고품질 데이터셋을 활용하여 정확도를 두 배로 향상시킵니다.
- 이미지 수준의 감지기와 메모리 기반 비디오 추적기를 공유하는 구조로 설계되었습니다.
- 새로운 SA-Co 벤치마크를 통해 프롬프트 기반 개념 분할을 지원합니다.

상세 설명

SAM 3(Segment Anything Model 3)는 이미지와 비디오에서 객체를 감지하고 분할하며 추적하는 통합 모델입니다. 이 모델은 "노란색 학교 버스"와 같은 짧은 명사구, 이미지 예시 또는 이 둘의 조합을 개념 프롬프트로 사용하여 작동합니다. 프롬프트 기반 개념 분할(PCS)을 통해 이러한 프롬프트를 입력하면 해당 객체의 분할 마스크와 고유한 아이디를 반환합니다. SAM 3 은 400만 개의 고유 개념 레이블을 포함하는 고품질 데이터셋을 생성.....

참조 자료

SAM 3 : Segment Anything with...
2025 · HuggingFace

★ New

MedSAM3 : Delving into Segment...
Anglin Liu, Xu R. Cao, ...2025

★ 128

UnSAMv2 : Self-Supervised...
Junwei Yu . 2025 GitHub.

★ 89

RAGAS 기반 성능 평가

Reranker 도입 전후(3rd vs 4th) 정량적 성능 비교 및 배포 효율성 분석

RAGAS SCORE



3rd Version

4th Version

Before vs After (Reranker)

✓ 측정 완료

METRIC	3RD	4TH	IMPROVEMENT
--------	-----	-----	-------------

논문 검색 정확도 Top-k Accuracy	87.5%	100%	+12.5%p 🎉
-----------------------------------	-------	------	-----------

Context Recall 정보 재현율	0.63	0.82	↑ 30%
--------------------------	------	------	-------

Context Precision 정보 정확도	0.90	0.98	↑ 9%
-----------------------------	------	------	------

Faithfulness 답변 사실성	0.94	0.95	↑ 1%
------------------------	------	------	------

Answer Correctness 답변 정확성	0.63	0.79	↑ 25%
------------------------------	------	------	-------



배포 시간

30분

1~3 min

Docker Compose

↓ 96% REDUCTION

복잡한 환경 설정 없이 단 2줄의 명령어로 배포 완료



향후 개발 계획 (Future Plan)

AI 성능 고도화, 데이터 확장, 인프라 안정화를 위한 단계별 전략



AI 성능 향상

검색 정확도 및 응답 속도 최적화

Multi-Query Retrieval

High Priority

하나의 질문을 여러 방식으로
바꿔서 검색 정확도 ↑

쿼리 임베딩 캐싱

Planned

같은 질문 반복 시 응답 속도
10배 향상



데이터 자동 수집 & 기능 확장

데이터 커버리지 및 사용자 편의성 확대

arXiv API 연동

Expansion

최신 AI 논문 자동 수집
(매일/매주 업데이트)

북마크 &

검색 히스토리

New Feature

사용자가 중요한 논문 저장
및 이력 관리



인프라 개선

대규모 트래픽 처리를 위한 시스템 안정성 확보

PostgreSQL

마이그레이션

Optimization

벡터 검색 최적화 + 동시 접속
효율적 지원

CI/CD 파이프라인 구축

DevOps

코드 푸시 시 자동 테스트
& 배포 자동화

PaperSnack 프로젝트 여정을 통해 얻은 성과와 교훈

🏆 핵심 성과

- ✅ **실무 수준 RAG 시스템 구축**
LangGraph 기반 고도화된 파이프라인
- ✅ **Docker 원클릭 배포 자동화**
배포 시간 96% 단축 및 환경 일관성 확보
- ✅ **마이크로서비스 아키텍처 경험**
Django(Web) + FastAPI(AI) 역할 분리
- ✅ **하이브리드 검색 + 재랭킹**
Cross-Encoder 도입으로 정확도 대폭 향상
- ✅ **RAGAS로 성능 검증**
정량적 지표 기반의 체계적인 품질 개선

💡 배운 점 (Lessons Learned)

Search First

"프롬프트 튜닝보다 검색 품질이 우선"
Garbage In, Garbage Out 원칙 재확인

Dockerize It

"내 컴퓨터에선 되는데..." 해결
컨테이너가 주는 개발/운영의 평화

Measure It

"측정할 수 없으면 개선할 수 없다"
지표 기반의 의사결정 습관화

Document It

"좋은 문서는 코드만큼 중요하다"
협업을 위한 명확한 스펙과 가이드

“

"Django + FastAPI를 사용하여 RAG 챗봇을 구현하고

Docker로 배포하는 과정이 쉽진 않았지만,

팀원들과 함께 성장하며 PaperSnack을 완성할 수 있었습니다."

Team 해조:

김지은

박다정

오학성

정소영

황수현

Thank you for your attention.